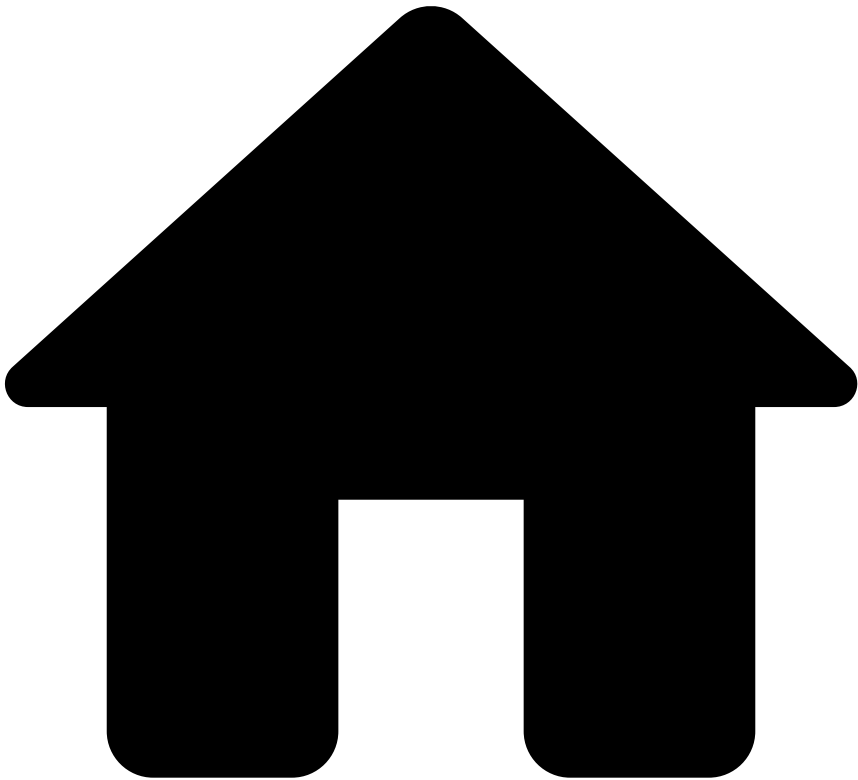


•



- Securing an Application
- Security Best Practices

On this page

Was this page helpful? 

Security Best Practices

This section covers some of the best practices for securing Pulse applications. It shows a sample security scenario and explains how each security entity helps keeping the application secure and maintainable. The page covers some generic security best practices you should pay attention to, even if your project organization differs from the sample scenario presented here.

Local data source permissions

By default, a Pulse user who has permissions to create data sources can access any file in the local filesystem of the Pulse server.

This means that by specifying `file://<path-to-file>` in the URL path, the user might be able to have the contents of sensitive files displayed in the data source preview. Examples include:

- `/etc/passwd`
- `/etc/shadow`
- Credentials to other machines such as databases that are present in config files under Pulse directory: `conf/`

For this reason, Pulse has a property that limits the allowed paths. Note that this property only applies locally, and thus, remote locations such as HDFS should restrict file accesses using the remote system's permissions, using the property `allowedLocalDatasourcePaths`.

Below is an example on how to restrict the allowed local paths of source file locations for data sources, assuming that Pulse is installed in the path `/opt/pulse`:

```
pulse.global.allowedLocalDatasourcePaths=/opt/pulse/var/datasources
```



info

The restriction on allowed local paths can also be specified per **Pulse app**.

Sample security scenarioâ

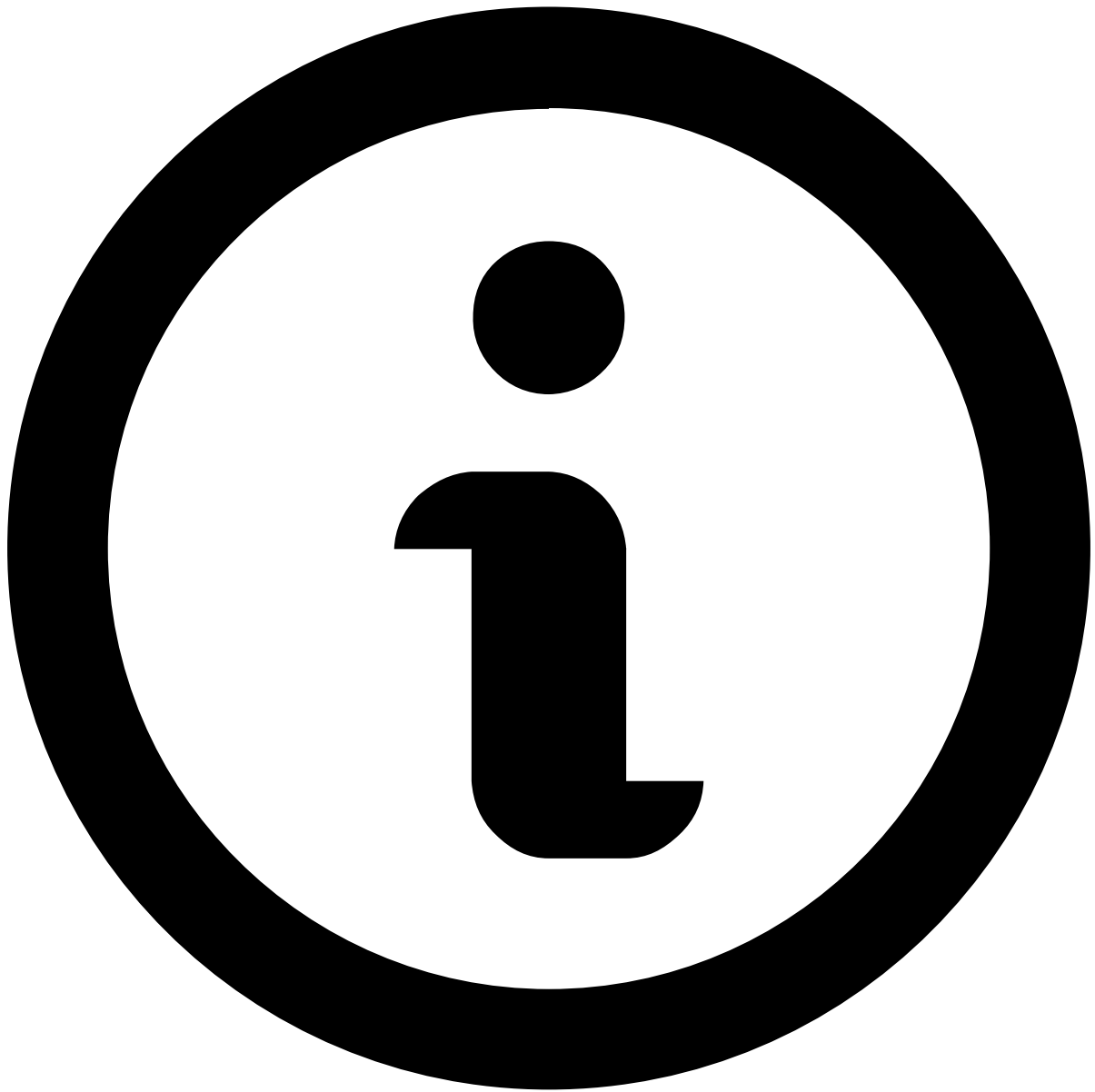
The following diagram depicts a hypothetical scenario of an acquiring company deploying Pulse to provide its merchants with fraud detection and prevention capabilities. All merchants will benefit from the acquirer defined fraud detection rules and each merchant's own fraud analysts can create customized rules tailored to their specific cases.

Security entities and their relationshipsâ

We will now look at each security entity, explain why it is needed and how it interacts with the other security entities. We will also analyze the role of each security entity on building our sample security scenario.

Usersâ

Users are the most well known security entity. They are present in almost every system with authentication. Every subject that authenticates in Pulse must have a unique user. The authentication of the user can be managed by Pulse itself or by an external authentication system such as Active Directory. Users are only responsible for the authentication part. Authorization is handled by permissions and resource rights. Users belong to groups which are the easiest and more maintainable way of managing authorization.

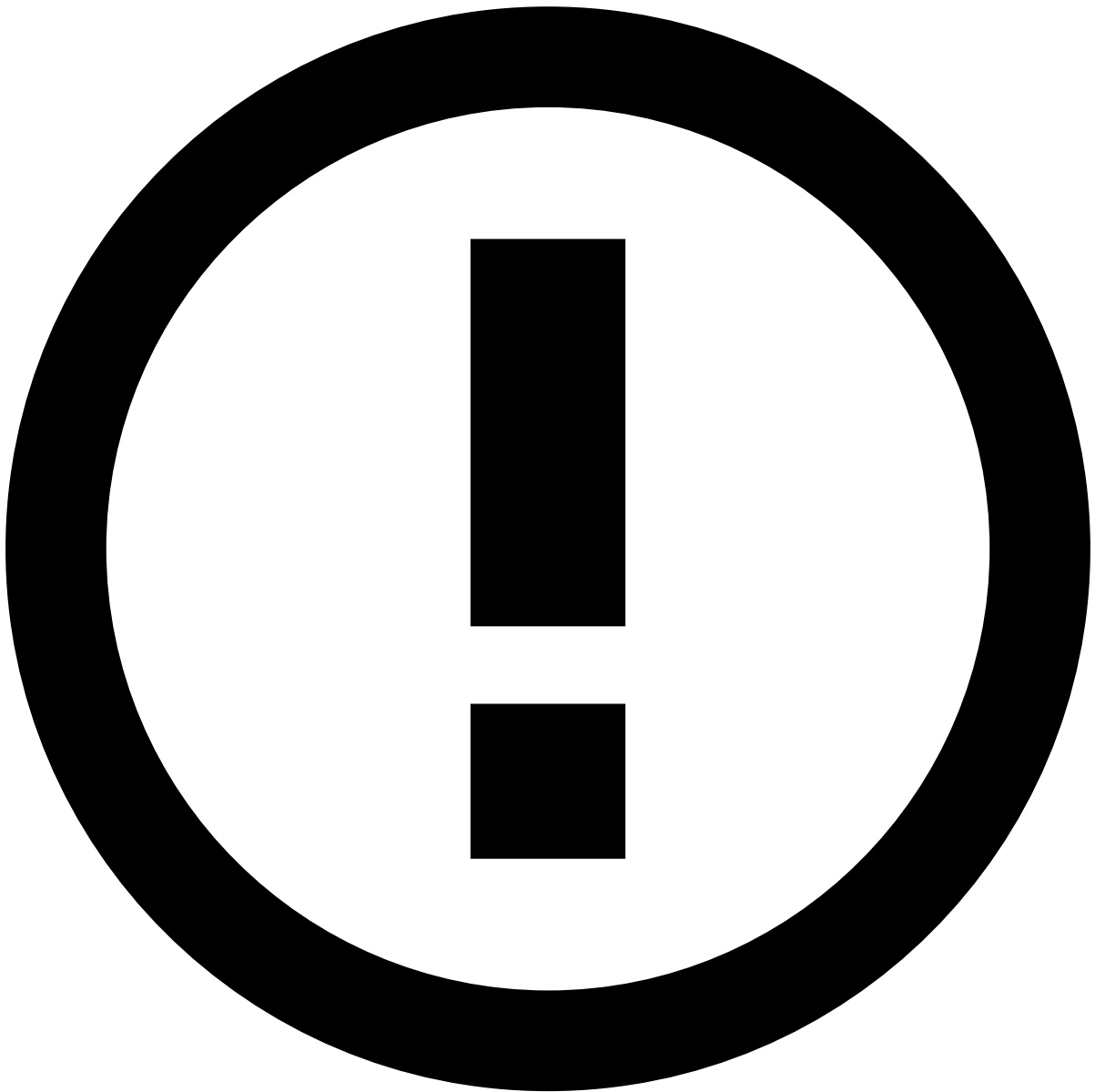


note

When creating new users, it is a good security practice to always set the default ownership. In practice, this will ensure that resources created by that user will, by default, be created with those chosen groups and rights for each group.

Groups

Groups represent organization positions that users of the system will belong to. In our sample scenario, **user Merchant B Employee 1 belongs to the groups Merchant B Employees and Merchant Employees**. We can say this user is an employee of Merchant B and also a generic Merchant employee.



info

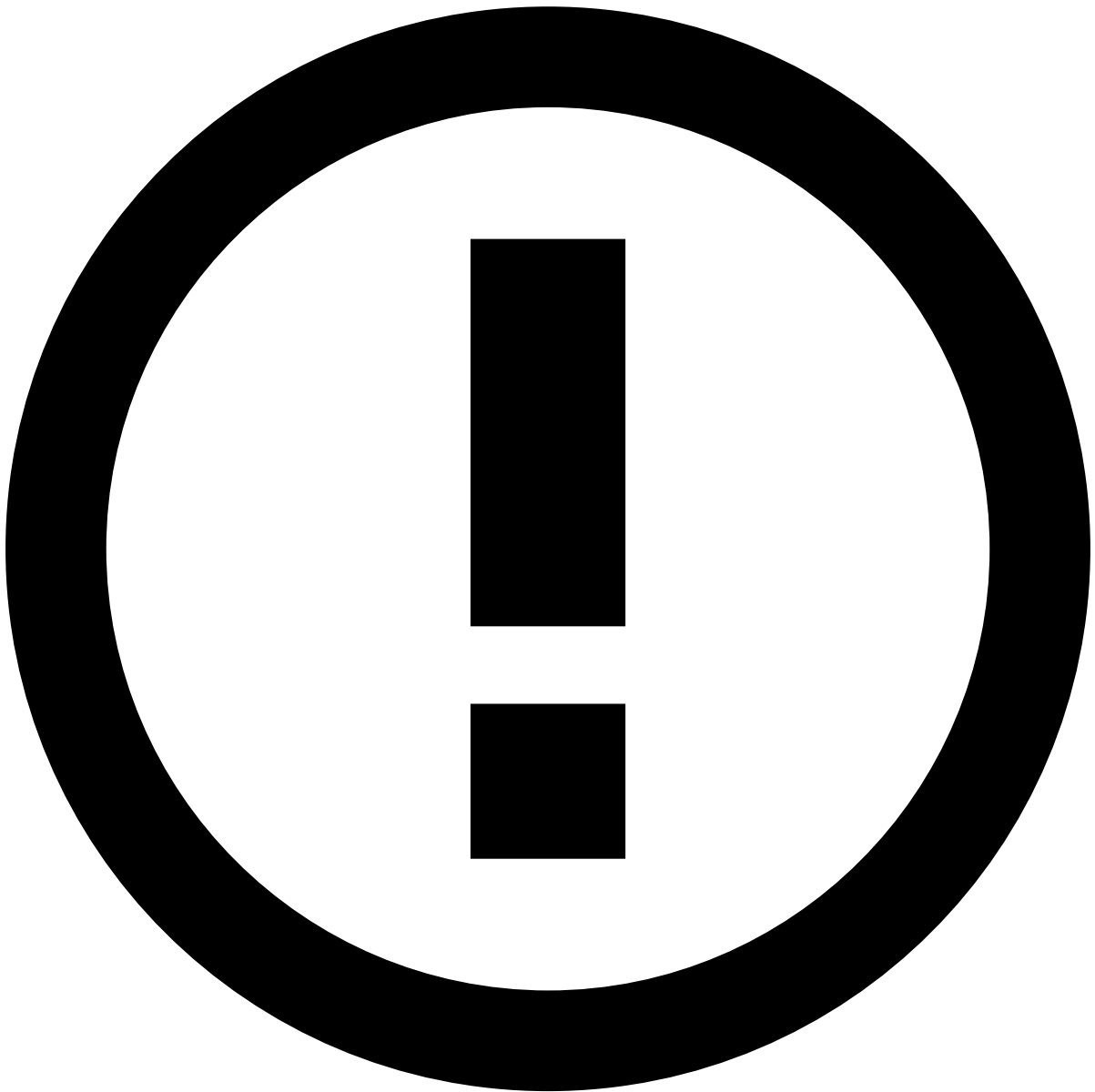
It is useful to have the user belong to a generic group of merchant employees because some of the system resources might have to be shared to all merchant employees regardless of the merchant. This greatly simplifies the maintenance of the system.

In our sample scenario, an example of a resource that should be shared to the *Merchants Employees* group is the *Generic Merchant Rules* resource. In this case, the sharing would be **read-only**, we will see how to do that later.

Each group might have zero or more associated roles. Having roles associated to groups is useful in several ways. It helps managing what each user can do by just assigning them to their groups, instead of having to give roles to each user. It also helps when you need to revoke some role from all users in a group, in this case you just have to edit the group, instead of editing all the users. Learn everything about groups in the [Security Entities](#) page.

Roles and permissions🔒

A [permission](#) is a right to do a very specific action. Pulse has many built-in permissions for managing the resources; for instance, the permission to *read fraud rules*, or the permission to *create new fraud rules*.



info

A permission is just a right to do some action on a resource (e.g. *read fraud rules*). It says nothing about the right to do that action on a particular resource instance (e.g. *read fraud rule X*). Resource rights are the topic of the next section.

A [role](#) is just a set of permissions.

Permissions are the most fine-grained security entity. After creating suitable roles, you will not have to use permissions directly. They will be enforced through the association of users to specific roles and groups. As we said earlier when discussing groups, roles can be associated to groups, making the group indirectly associated with the permissions in those roles. Groups are the highest level security entity and you should use them to create secure and maintainable deployments.

In our example scenario, both *Merchants Managers* and *Merchants Employees* groups are associated with roles. The roles depicted here do not exist in Pulse and have to be created with the relevant permissions. The *rules-management* role might give users of the *Merchants Managers* group permissions to create, read, update and delete fraud rules. While a user of the *Merchants Employees* group might only have permissions to manage templated fraud rules and read fraud rules.

Learn everything about roles and permissions in the [Security Entities](#) page.

Resource rights ⓘ

Resource rights dictate what users of a group can do to a resource instance.

Resource rights are granted when the resource instance is created or updated. To understand this section you should be familiar with the concepts covered in the [Securing Metadata](#) page.



info

Resource rights never override the user's permissions. If a user does not have the permission to read fraud rules, a fraud rule shared with that user's group will still be invisible to the user.

Considering our sample scenario, *Merchant A Rules* and *Merchant B Rules* must be visible only to users of *Merchant A Employees* and *Merchant B Employees* groups respectively. To achieve this, the user creating the rule must share it with the group and grant that group the desired rights. In this case it could be the rights to do everything with the rule, since this would apply only to users of that group. The *Generic Merchant Rules* should be shared with the *Merchants Employees*

group but only having *Read* right. Remember that users belonging to groups higher on the groups hierarchy will have the same rights, if not more.

Read the [Typical permission setups](#) for guidance on specific cases related to rights management.

Use cases

This section covers some of the best practices related to common security use cases. Some are related to the sample scenario presented before while others are more generally applicable.

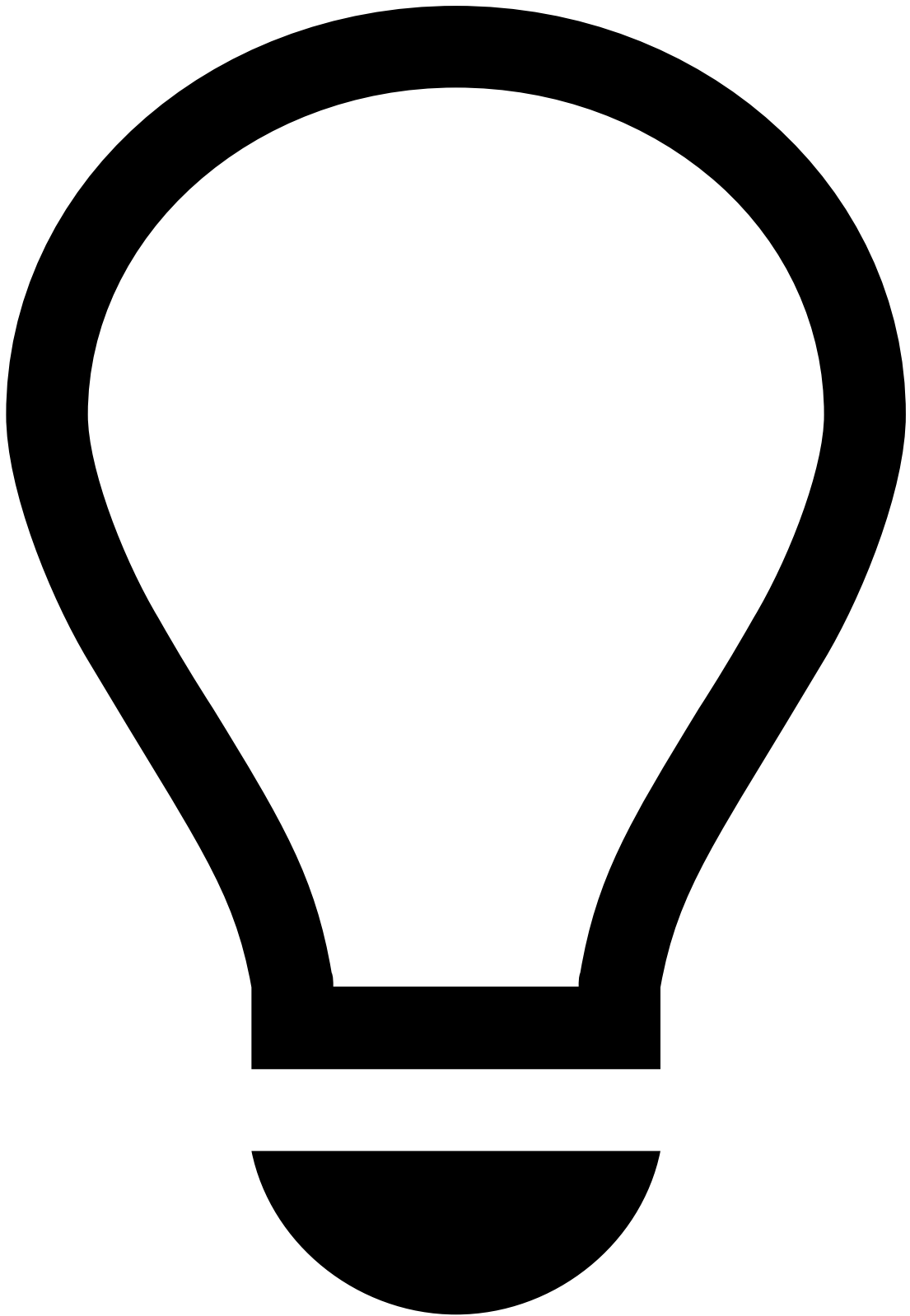
Restricting the manage right on applications

When creating an application remember to configure the application ownership to Groups and grant the *manage* right only to a restricted group of application administrators. This will prevent any user with access to the application from inadvertently or maliciously publishing or stopping the application.

Creating a new user

When creating new users, there are no default assignments regarding groups, roles, etc. The administrator must know beforehand how to configure the new user for it to reach its goals. Usually adding users is not time consuming, since it's quite common to already have some roles completely configured, hence it's only a matter of connecting the new user to the designated groups and it's done.

So let's say you need to create a user that manages all the merchants - let's call it *Merchant Manager 1*. If you assign this user to the *Merchants Managers* group, it will have access to all the groups that are below in the group hierarchy - that is *Merchant A**,** *Merchant B** and *Merchants Employees* - besides all resources and roles assigned to the original assignment group (*Merchants Manager*). All the resources and roles bound to these groups will be available as well to this new user.



tip

The higher a user is in the groups hierarchy, the more rights will accumulate and therefore more power over the application resources.